CS5340 Assignment 4:
Monte Carlo Inference

1 Overview

In this assignment, you will implement two Monte Carlo inference methods: importance sampling and Gibbs sampling. Marginalization over large or continuous probability distributions can be intractable. Monte Carlo methods instead use samples to approximate the required target distribution, making inference more tractable for complex models.

Reference: Lecture 9

Honour Code. This coding assignment constitutes **15%** of your final grade in CS5340. Note that plagiarism will not be condoned! You may discuss with your classmates and check the internet for references, but you **must not** submit code or a report copied directly from other sources.

2 Submission Instructions

Submit one zip archive containing the following:

- **Source code:**
  - `part1/main.py` – code for importance sampling.
  - `part2/main.py` – code for Gibbs sampling.
- **Report (`report.pdf`).** Describe your implementation in no more than one page.

Indicate clearly your name and student number (the one that looks like `A1234567X`) in the report and at the top of your source code. Name the archive `A1234567X_lab4.zip`, replacing the example with your student number.

Submit your assignment by **1 November 2026, 2359HRS** to Canvas. 25% of the total score will be deducted for each day of late submission.

3 Getting Started

This assignment requires Python 3.5 or later. Install the required packages using:

```
pip install -r requirements.txt
```

If you have installation issues, post them in the forum so that other students or the instructors can help accordingly.

4 Test Cases

Sample inputs are in `data/inputs`, ground-truth files are in `data/ground-truth`, and your predictions will be written to `data/predictions`. Cases 1 and 2 use the Lecture 9 example shown in Figure 1. The remaining cases include a larger graph, evidence-variable variants, and a non-binary Part 1 example. Five validation cases are supplied for Part 1 and four for Part 2.

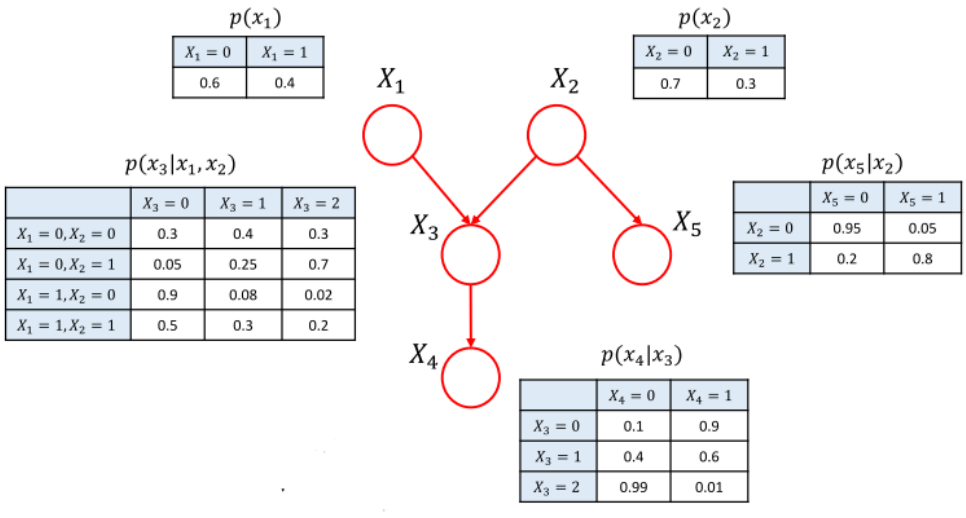


Figure 1: Bayesian network and local conditional probability tables used in validation cases 1 and 2.

The ground truth may differ slightly from your answers because the algorithms are stochastic. We use a tolerance of one decimal place. During grading, your code will be evaluated on hidden cases in addition to the supplied validation cases.

Your implementation must also run in a reasonable amount of time. As a reference, the original implementation produced the following timings:

Case	1	2	3
Importance sampling	13s	4s	37s
Gibbs sampling	12s	2s	39s

Table 1: Reference timings on an Intel i7-6500U at 2.50GHz.

Any runtime within approximately two to three times the reference timing is reasonable. For fairness, submissions will be compared on the same system.

5 Importance Sampling

In Part 1, you are given local target conditional distributions $p_u(x_u \mid x_{\pi_u})$ and local proposal conditional distributions $q_u(x_u \mid x_{\pi_u})$, where π_u denotes the parents of node u. The joint proposal and target distributions factorize as

$$q(\mathbf{X}) = \prod_u q_u(x_u \mid x_{\pi_u}), \qquad p(\mathbf{X}) = \prod_u p_u(x_u \mid x_{\pi_u}). \tag{1}$$

Each sample is weighted by $p(\mathbf{X})/q(\mathbf{X})$. Return the approximate conditional distribution $p(\mathbf{X}_F \mid \mathbf{X}_E)$, where $\mathbf{X} = \mathbf{X}_F \cup \mathbf{X}_E$, $\mathbf{X}_F$ contains the query nodes, and $\mathbf{X}_E$ contains the evidence nodes.

Sample nodes in topological order: every parent must be sampled before its children, and a parent's sampled value must be used when sampling a child.

Implement the following functions:

- **`_sample_step()` [3 points].** Perform one complete sampling iteration.
  - Sample separately from each local proposal distribution; do not sample from a precomputed joint proposal distribution.
  - For Figure 1, X_1, X_2, X_3, X_4, X_5 is one valid topological order.
  - Update the current assignment immediately after sampling a parent.
- **`_get_conditional_probability()` [4 points].** Perform N iterations, weight the samples, and return $p(\mathbf{X}_F \mid \mathbf{X}_E)$.
  - Do not construct joint proposal or target factors. Compute each sampled value of $p(\mathbf{X})$ and $q(\mathbf{X})$ as the scalar product of the relevant local factor entries.
  - Incorporate evidence variables when determining the graph and sampling order.

Hint: You may add helper functions between the designated comment blocks in each file.

6 Gibbs Sampling

In Part 2, for each node x_u you are given a conditional distribution

$$q_u(x_u \mid \mathbf{X} \setminus \{x_u\}). \tag{2}$$

Sample from each node's conditional distribution while holding the remaining nodes fixed. All retained samples are weighted equally to approximate $p(\mathbf{X}_F \mid \mathbf{X}_E)$.

Implement the following functions:

- **`_sample_step()` [3 points].** Perform one full Gibbs sweep, following the procedure in the Lecture 9 example.
- **`_get_conditional_probability()` [5 points].** Perform N iterations and return the approximate conditional distribution.
  - Reduce the supplied distribution for each node to its Markov blanket before sampling.
  - Incorporate the supplied evidence variables into the graph and keep their values fixed throughout sampling.

Hint: Helper functions may be added within the designated blocks; efficient implementations may receive credit.